

A Project Report On

Intelligent Intrusion Detection System: Detecting Cyber Threats with Machine Learning

*A major project phase-I that served to complete partial academic requirements for the
award of the degree of*

**BACHELOR OF TECHNOLOGY
IN
INFORMATION TECHNOLOGY
(2022-2026)
BY**

R. PRATHEEK REDDY	22241A12B7
T.LOK VINAY	22241A12C3
P. DINESH REDDY	22241A12B5

*Under the Esteemed Guidance
of*
MUNAGOTI USHA RANI
Assistant Professor



**DEPARTMENT OF INFORMATION TECHNOLOGY
GOKARAJU RANGARAJU INSTITUTE OF ENGINEERING AND TECHNOLOGY
(AUTONOMOUS)
HYDERABAD
2025-26**



CERTIFICATE

This is to certify that it is a Bonafide record of Major Project Phase - I work entitled “**Intelligent Intrusion Detection System: Detecting Cyber Threats with Machine Learning**” done by **R.PRATHEEK REDDY (22241A12B7), T.LOK VINAY(22241A12C3), P.DINESH REDDY(22241A12B5)** of **B. Tech** in the Department of Information of Technology, **Gokaraju Rangaraju Institute of Engineering and Technology** during the period 2022-2026 in the partial fulfillment of the requirements for the award of degree of **BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY** from GRIET, Hyderabad.

MUNAGOTI USHA RANI

Assistant Professor,
(Internal Guide)

Dr. Y. J. Nagendra Kumar

Head of the Department

(Project External)

ACKNOWLEDGEMENT

We take the immense pleasure in expressing gratitude to our Internal guide, **Mrs. MUNAGOTI USHA RANI**, Assistant Professor, Dept of IT, GRIET. We express our sincere thanks for her encouragement, suggestions and support, which provided the impetus and paved the way for the successful completion of the project work.

We wish to express our gratitude to **Dr Y.J. Nagendra Kumar**, HOD IT, and our Project Coordinator, **Mr. P.K. Abhilash**, for his constant support during the project.

We express our sincere thanks to **Dr. Jandhyala N Murthy**, Director, GRIET, and **Dr J. Praveen**, Principal, GRIET, for providing us with a conducive environment for carrying through our academic schedules and projects with ease. We also take this opportunity to convey our sincere thanks to the teaching and non-teaching staff of GRIET College, Hyderabad.



Email: pratheekreddyreddammagari18@gmail.com

Contact No: 6304265122



Email: thiruveedhulaloki@gmail.com

Contact No: 8886665093



Email: dineshreddy200419@gmail.com

Contact No. 6302555344

DECLARATION

This is to certify that the major project phase-I entitled “**Intelligent Intrusion Detection System: Detecting Cyber Threats with Machine Learning**” is a Bonafide work done by us in partial fulfilment of the requirements for the award of the degree **BACHELOR OF TECHNOLOGY IN INFORMATION TECHNOLOGY** from Gokaraju Rangaraju Institute of Engineering and Technology, Hyderabad.

We also declare that this project is a result of our own effort and has not been copied or imitated from any source. Citations from any websites, books and paper publications are mentioned in the References. This work was not submitted earlier at any other University or Institute for the award of any degree.

R. PRATHEEK REDDY 22241A12B7

T.LOK VINAY 22241A12C3

P. DINESH REDDY 22241A12B5

ABSTRACT

In the current digital environment, network security is a significant issue, and cyberattacks are becoming more numerous by the day. Conventional security systems, such as firewalls and antivirus software, have not been useful in most cases to detect sophisticated dynamic attacks. In this project, a proposal is presented for a Machine Learning-based IDS, which can detect and classify network traffic efficiently, whether normal or intrusive.

It uses the NSL-KDD dataset as a reference dataset to study intrusion detection. Normalization, encoding categorical variables and feature selection are the three key data preprocessing processes. To this end, an XGBoost classifier was trained on the 10 most significant network features, selected to make trade-offs between the ease with which it would run in real-time and the maximum possible accuracy.

It performs the training in a training model, uses a FastAPI server to process the prediction requests, and provides a React-based frontend to enable users to feed values into the features via an interactive application. This frontend communicated with the backend by making REST API calls and shows the result as either normal or intrusion detection, and in a visual form of pie and bar charts to make it easier to interpret.

The development of the proposed IDS has demonstrated an efficient and scalable system to record network intrusions with enhanced detection rates and lower complexities.

Keywords: Intrusion Detection System, Cybersecurity, Machine Learning, NSL-KDD, Flask, Real-time Detection, Network Security, Python, Scapy.

TABLE OF CONTENTS

	Name	Page no
	Certificates	ii
	Abstract	v
	Contents	vi
1	INTRODUCTION	1
1.1	Introduction to the project	1
1.2	Problem Statement	2
1.3	Main Objectives	2
1.4	Scope of the Project	3
1.5	Technologies Used	4
2	LITERATURE SURVEY	5
3	SYSTEM ANALYSIS	10
3.1.1	Functional Requirements	10
3.1.2	Non-Functional Requirements	10
3.2	Feasibility Study	10
4	SYSTEM DESIGN	11
4.1	UML-Diagrams	11
4.2	Architecture Diagram	13
4.3	Use Case Diagram	14
4.4	Class Diagram	15
4.5	Sequence Diagram	16
4.6	Activity Diagram	17
4.7	Component Diagram	18
4.8	Deployment Diagram	19
5	IMPLEMENTATION	20
6	SOFTWARE TESTING	30
6.1.1	Unit Testing	30
6.1.2	Integration Testing	31
6.1.3	Functional Testing	31
6.1.4	Performance Testing	32
6.1.5	Security Testing	32

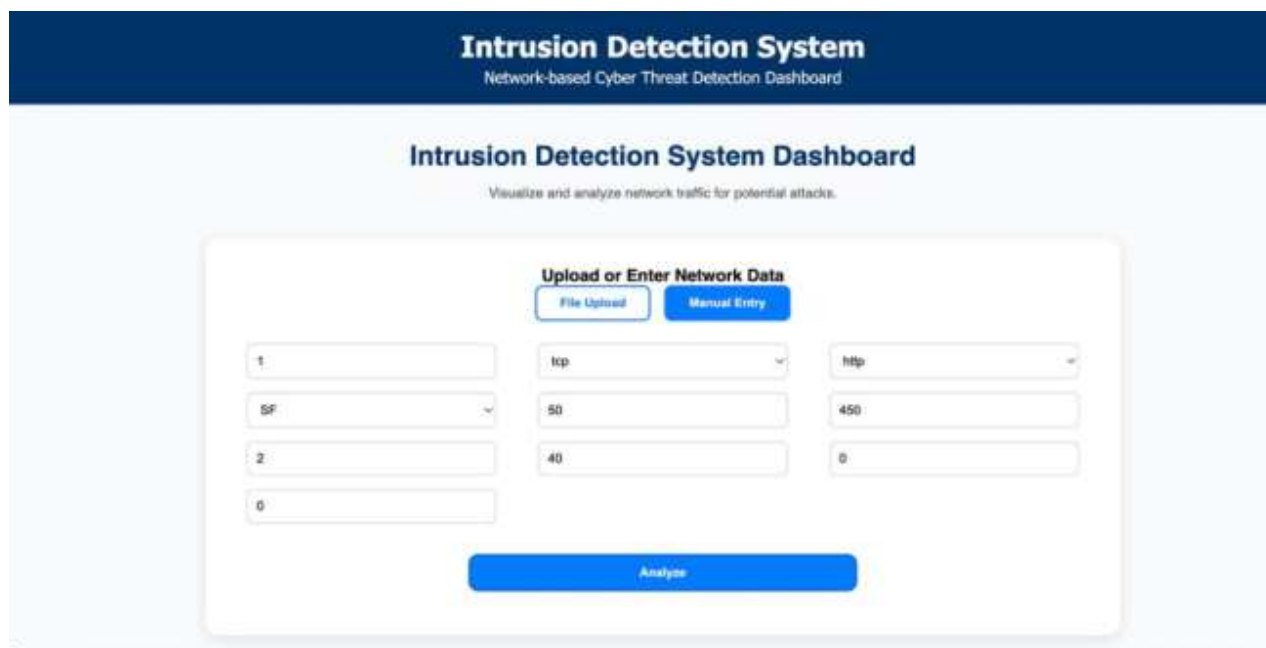
6.2	RESULTS	33
7	CONCLUSION AND FUTURE SCOPE	36
8	REFERENCES	37
9	LIST OF DIAGRAMS	
S No	Figure Name	Page no
1	Architecture Diagram	13
2	Use Case Diagram	14
3	Class Diagram	15
4	Sequence Diagram	16
5	Activity Diagram	17
6	Component Diagram	18
7	Deployment Diagram	19

1. INTRODUCTION

1.1 Introduction to Project

This digital era has led to the proliferation of cyberattacks to the extent that this is a real basis of concern to organizations, governments, and individuals. Technology continues to change, and so do the methods of attack whereby attacks are currently more technologically advanced than what the traditional security measures of physical or virtual firewalls and antivirus software can manage/detect. It is extremely difficult to track these dynamic patterns of intrusion in these conventional systems to ensure the safety of sensitive data and critical networks.

It is against this background that we are proposing a project, namely, a Machine Learning-Based Intrusion Detection System. The main aim of the project is to develop a high-quality and smart system that employs machine learning to identify the network traffic as normal or intrusive. The system is based on the XGBoost classification, data preprocessing, and interactive visualization of the data in a Single location such that it is both accurate and usable. The concept behind this project is to create a secure, adaptive tool that will help in the process of securing the digital environment by detecting intrusion in real-time and making decisions in advance concerning cybersecurity.



[FIG-1] Project Dashboard

1.2 Problem Statement:

- **Growing Cyber Fraud:** More frequent and advanced cyber-attacks have been experienced due to advanced networked systems that take advantage of weak points that traditional security controls can hardly or impossible detect.
- **Lack of applicability of Traditional Security Tools:** Firewalls and antivirus programs rely on a set of rules or known signatures; therefore, they are rather inefficient when it comes to new or changing attack patterns.
- **The absence of Intelligent Detection:** The majority of the available systems are not able to learn from previous attacks or detect latent patterns in the network traffic, thus they are highly susceptible to false negatives and false positives.
- **Real-time Analysis:** The volume of network traffic may not always be investigated using the traditional methods of detection in real-time; it postpones the discovery of possible intrusions.
- **Absence of an Integrated Smart Solution:** There is a desperate need to have an integrated, easy-to-use platform with machine learning, on-the-fly prediction, and graphical analytics, which can classify the network behaviour accurately to enhance cybersecurity.

1.3 Principal objectives:

- **Detection based on Machine Learning:** To identify normal and intrusive traffic within the network, create an intelligent model, utilising the XGBoost algorithm, and based on significant network characteristics.
- **Data Preprocessing and Feature Selection:** Take care of encoding, normalization and feature selection, to enhance model efficiency to achieve better detection accuracy.
- **Interactive Web Interface:** React.js frontend to serve the following purpose: to enter the network parameters and present the results of the classification in a simplified form.
- **Integration:** Use FastAPI to communicate with the frontend using the model and Fast API to make ML predictions and communicate effectively using RESTAPI.

- **Prediction results and performance metrics:** Visualization. The performance metrics and prediction results need to be presented as interactive visualizations and plots to make them understandable.
- **Live and Sizable System:** The IDS must be scalable to operate in live systems, and it must be scalable to function on a broader network.

1.3.1 Input Parameters:

The input parameters to the system that the system uses to analyse and classify network traffic as normal or intrusive include:

- **Duration:** This is the length of the network connection session.
- **Protocol Type:** Type of network protocol used, e.g. TCP, UDP, and ICMP.
- **Service:** The name of the network service or port through which the connection is made.
- **Flag:** It is the state of the connection; is it normal or in error?
- **Source Bytes:** This is the amount of data that the source transmits to the destination.
- **Destination Bytes:** This is the number of data that is transmitted between the source and the destination.
- **Count:** This is the number of connections with the same host within a particular time window.
- **Same Host Rate:** The percentage of the connections to the same host as the current one.
- **Same Service Rate:** Percentage of the connections that are using the same service.
- **Error rate:** Percentage of connections that end in error or abnormal termination.

1.4 Scope of the Project:

- **Smart Threat Detection:** The network traffic patterns are classified by the system to identify suspicious activities with the help of the XGBoost machine learning technique in real time.
- **Enhanced Security Visibility:** Provides the user with analysis about network behaviours to identify possible vulnerabilities and a set of attack trends.
- **User-friendly:** It has an easy-to-use React.js frontend, in which the user is allowed to input the parameters and view the results easily in an intuitive, interactive interface.

- **Visualization and Reports:** This feature incorporates the use of graphical visualizations, such as pie, bar charts, and similar, to simplify the understanding of intrusion patterns.
- **Scalable and Real-Time Processing:** The system is developed using FastAPI, and it provides the system with high performance, scalability, and a seamless interface between the model and the frontend to process intrusion in real time.
- **Modular Data Mining Framework:** This data mining framework design will be modular such that additional algorithms, datasets, and visualization modules can be added in future releases.

1.5 Technologies Used:

- **Frontend:** React.js is a quick and efficient JavaScript library that was used to build a responsive and dynamic user interface.
- **Backend:** It is developed with FastAPI, which is a modern and lightweight Python platform that allows asynchronous computing and easy interactions between the API and other services.
- **Machine Learning:** This uses the XGBoost Classifier to classify and categorise network data as either normal or intrusive.
- **Visualization Library:** This is an option that employs Chart.js (or other similar libraries) to use interactive graphical dashboards to show the results.
- **Dataset:** It uses the NSL-KDD dataset, which is a well-known network intrusion detection research benchmark dataset.
- **Model Deployment:** It involves the trained ML model combined with a FastAPI backend to make predictions to the frontend by using REST APIs.

2. LITERATURE SURVEY

- Introduction Kathryn-Ann Tait, Jan Sher Khan, Fehaid Alqahtani, Awais Aziz Shah, Fadia Ali Khan, Mujeeb Ur Rehman, Wadii Boulila, and Jawad Ahmad (2021) carried out a study on Intrusion Detection with Machine Learning Techniques: An Experimental Comparison. The paper has compared different machine learning algorithms to IDS and evaluated the algorithms in terms of detecting network intrusions. The study technique was an experimental comparison of various classifiers over standard intrusion detection data sets. The findings were very accurate in binary and multiclass attacks, in terms of accuracy and differentiation, respectively. Nevertheless, the research also emphasized the problem of the most suitable choice of algorithm, and it was observed that the results were not the same when using various datasets [1].
- The article by MA Kumaar et al. (2022) suggests a Hybrid Framework of Intrusion Detection in Healthcare. In this study, a deep learning hybrid immune framework based on Immune Net was proposed for use in healthcare network IDS. The system advanced the identification of the current intrusion forms, and offered better security to sensitive healthcare information, especially. Although this model demonstrated some good results, it was very demanding in terms of computational resources; in other instances, it demanded specialized hardware to make it effective [2].
- The article by Z. Li (2024) entitled Towards Deep Learning based Intrusion Detection System discusses the application of deep learning in the creation of a smart IDS infrastructure. The study points to the possibility of using convolutional and recurrent neural networks to establish the most current cyber threats with a more precise determination. The focus on deep learning models is attributed to their capacity to accommodate novel attack patterns and, hence, enhance the stability of the network security systems. The system, however, requires a massive labelled dataset and requires computational power to be trained and deployed in real-time, thus limiting its scalability in resource-constrained environments [3].

- B. Buyuktanir et al. (2025) publish the paper Federated Learning in Intrusion Detection: Advancements, Challenges, and Future Directions, which is structured according to an extensive overview of federated learning in the context of intrusion detection systems. This study indicates that federated models improve user privacy through the localisation of training data on distributed devices and minimize the transmission overhead. The strategy is used to maintain safe cooperation between networked systems without prejudice to sensitive information. However, problems like the complexity of model aggregation and data heterogeneity are very difficult to apply in a large-scale network. [4].

- The article Hybrid Deep Neural Networks to Intrusion Detection by R. Kumar and M. Singh (2024) presents a hybrid model of intrusion detectors, consisting of Convolutional Neural Networks (CNN) with Long Short-Term Memory (LSTM). The architecture provides a more accurate detection with fewer false alarms as it captures both space and time features on network traffic. The research uses benchmark data such as NSL-KDD and CICIDS2017 to assess the performance. Although the model has a good capability of detection, it is highly computationally intensive and lacks adaptability to real-time performance when using a high-speed network setup. [5].

- The article by L. Zhao and P. Chen (2023), Anomaly Detection in Network Security using Autoencoders and Feature Fusion, is devoted to using unsupervised deep learning techniques in intrusion detection. Using stacked autoencoders and feature fusion mechanisms, the system can easily draw the distinction between normal and malicious behaviors without the need to have labelled data. This increases responsiveness to attacks of the zero-day. Nevertheless, this accuracy can be reduced when it is employed in heterogeneous network environments because of the unbalanced feature representations.[6].

- The article Intrusion Detection Framework using Blockchain to ensure the IoT Networks are secure by S. Ahmed and R. Patel (2025) combines the idea of blockchain technology and IDS in order to enhance the state of transparency and data immutability. The decentralised architecture guarantees the safety and management of trust throughout communication across IoT devices. The system improves accountability and eliminates single points of failure that are associated with centralized IDS designs. On the downsides, blockchain structure implies latency and performance overhead, which might not be viable with small IoT nodes. [7].

- The article by H. Lee and D. Park (2024) Transfer Learning Approaches to Network Intrusion Detection addresses the topic of pretrained deep models and domain adaptation techniques in the development of an IDS. Transfer learning minimizes the use of large labelled datasets and speeds up the convergence of the models. It also facilitates cross-domain detection; thus, the system is applicable in various network settings. Nevertheless, the differences between source and target data distributions can result in a decline in performance, and the fine-tuning can be used to reach optimal generalization. [8].

- Moreno and Gupta (2023) suggest a lightweight CNN-based system of edge and Internet of Things intrusion detection. Their model does not need much computational power while still maintaining the same detection accuracy, resulting in its applicability in low-power settings. To make a fuller defense, they might need more frequent updates or more hybrid support since the simplified architecture might not be capable of withstanding the very sophisticated attacks [9].

- According to Sharma and Rodrigues (2024), explainable artificial intelligence (XAI) can improve the current intrusion detection systems by ensuring a greater transparency of their model decisions. Using techniques such as SHAP and LIME, analysts may rely on AI-based IDS and improve their perception of alerts. Although it has its advantages, XAI may raise the processing overhead and may not even be able to interpret quite deep models fully. [10].

2.1 Existing System:

The existing network security solutions mostly depend on the conventional defence techniques like firewalls, antivirus software and signature-based intrusion detection systems. These tools are effective against counterattacks, but they fail to identify zero-day attacks, polymorphic malware, and abnormal network behaviours. The majority of traditional systems examine packet headers or match signatures against threat databases; therefore, they are reactive rather than proactive.

❖ **Signature-Based IDS:**

- **Benefits:** Rapid recognition of established patterns of attacks at minimal levels of false positives.
- **Limitations:** Fails to capture new or modified attacks as it relies on preexisting signatures.

❖ **Anomaly-Based IDS (Rule-Driven):**

This category of IDS is known as the rule-driven anomaly-based IDS (ABIDS).

- **Technology:** Can detect behaviour by the network even in situations where the attack signature is unknown.
- **Limitations:** It causes a large number of false alarms because of the fixed definitions of thresholds and low adaptability.

❖ **Manual Monitoring Systems:**

- **Pros:** The security experts can evaluate the complex events according to human reasoning.
- **Limitations:** Not applicable to large networks and extremely time-consuming when analysing the data in real time.

2.2 Proposed System:

The suggested system presents a Machine Learning-based Intrusion Detection System (IDS) that is expected to identify known and unidentified cyber threats with more accurate results. It uses an XGBoost classifier that is trained on the NSL-KDD dataset, which is the refined benchmark dataset that is popularly used in intrusion detection studies. The model is centered around the 10 most important key network features, with a focus on both performance and computer efficiency.

The system also has an API based on FastAPI, which receives prediction requests and calls the trained model and classifies network traffic as either Intrusion or Normal. React.js front-end provides an interactive platform where the user may enter the values of the features and see the detection-in-real-time results in pie and bar charts.

This model is dynamic as opposed to rule-based systems, which do not change over time, but learns and become more accurate. It is scalable, easy to deploy and has a smooth user experience due to its integration with modern web technologies. In general, this IDS proposal fills the gap between machine learning intelligence and reality in the field of network security by offering an intrusion detection platform that is accurate, user-friendly, and in real time.

3. SYSTEM ANALYSIS

3.1.1 Functional Requirements:

- Users connect to the interface, and this requests intrusion detection after handling network traffic data.
- The FastAPI backend is a real-time data processing engine that operates based on the communication between the React frontend and FastAPI backend platforms via REST API.
- The XGBoost classifier will recognize the input provided as an intrusion or normal training and answer immediately using the interface.
- The system offers the basis of future upgrading, which comprises automated monitoring services, real-time network tracking, and enhanced visualization services.

3.1.2 Non-Functional Requirements:

The system is capable of fulfilling the criteria of trustworthiness and dependability in its operations.

- Fast system response is not more than three seconds with inbuilt scalability that accommodates multiple users with a 99 per cent availability of uptime.
- The API keys of the system, along with safe mechanisms of transferring data through the use of secure HTTPS, provide network data safeguards as well as an easy user interface.

3.2 Feasibility Study:

- The system consists of a combination of React as a front-end component and FastAPI (Python) as a back-end component with the XGBoost machine learning algorithm to classify the dataset of the NSL-KDD efficiently.
- The cost of low-maintenance and open-source tools is economically efficient because of the solution. The system works as a platform that is device-independent and can be easily utilized by the users and easily maintained.
- The system works under the regulations of data protection, along with the open-source framework licensing.

4. SYSTEM DESIGN

CONCEPT OF UML:

Unified Modelling Language (UML) is a well-defined language used to describe software systems and their components. This tool provides multiple diagrams and symbol options to display the entire system classification and component relationships, and specific subsections. Engineers apply the tool for structural and docket representation of any software development project to share design details among project team members.

Detailed design of complex software systems receives support from UML by establishing programming languages understood by developers and stakeholders, and analysts. The tool offers support for object-oriented design and excels at software engineering fundamentals, and serves as an effective tool for project planning and system structuring.

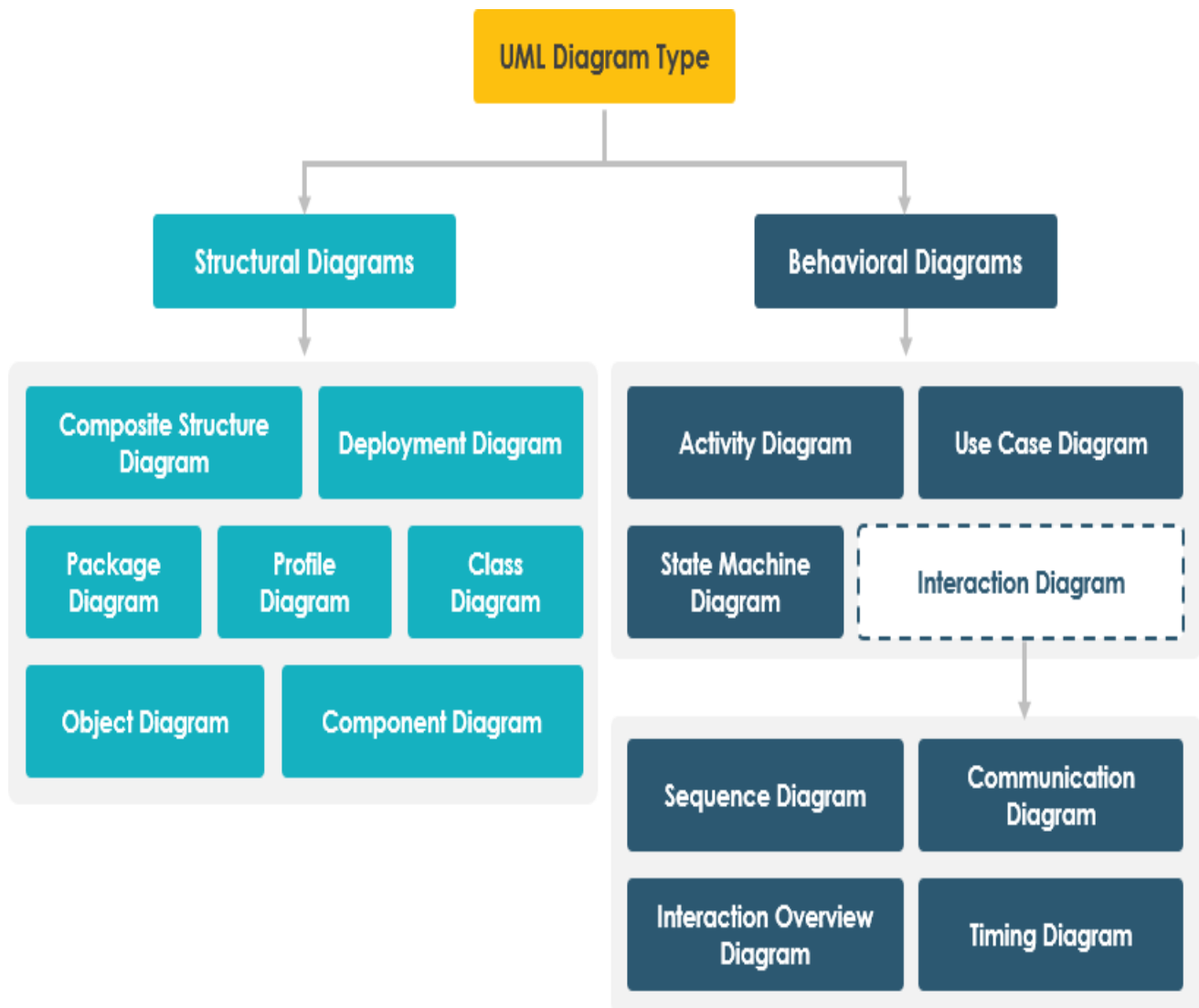
4.1 - UML DIAGRAMS:

UML represents a platform-independent modelling language that supports any programming language while working with different development methodologies, thus its standard nature makes it known to most developers. The increase of value with each additional diagram makes work workflow and pace more positive, although engineers typically show resistance to diagram use in Agile development. UML diagrams should be part of your documentation system and not just pretty extras. UML diagrams serve multiple functions to support engineering teams. The following benefits enable engineering teams when they use UML diagrams:

A UML diagram assists the team with developer onboarding, either from within existing members or external developers who move to a new team at the company.

- The proper behaviour for dealing with source code needs explanation.
- A project builds new features before beginning the programming stage.
- UML diagrams facilitate better information understanding when technical staff meet non-technical staff during observations.

- Diagrams in UML can be broadly classified...

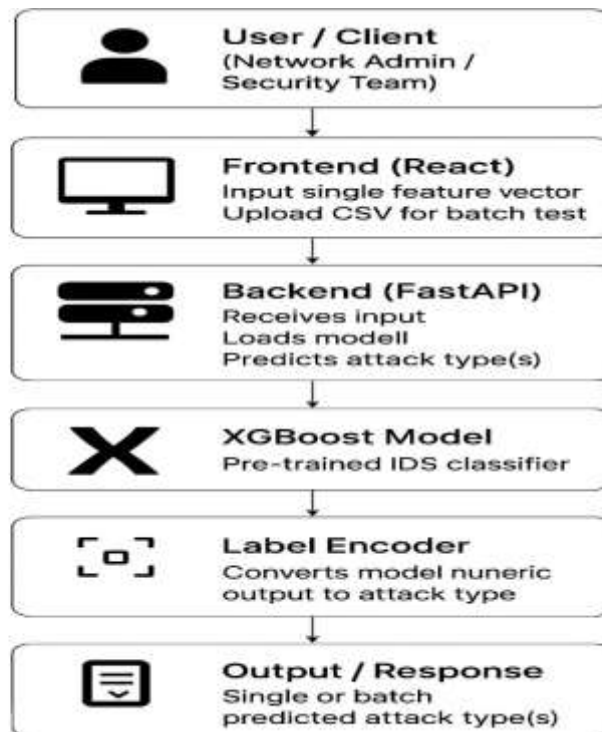


[FIG-2] UML Diagram Type

Concepts of UML

4.2 Architecture Diagram:

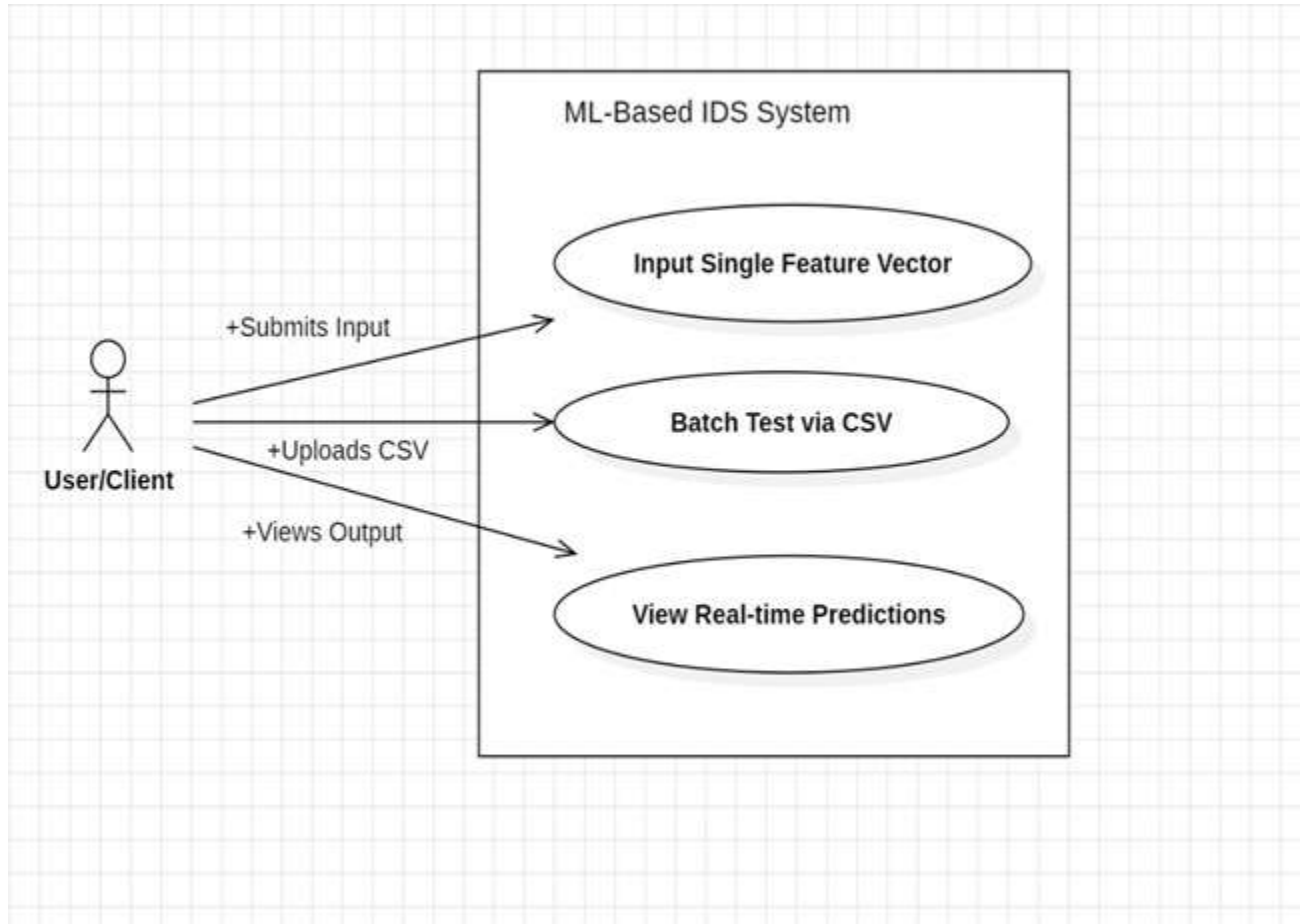
A well-designed frontend interface is a system that links the user input data to real-time network analysis and smart detection results. The frontend is developed on the basis of React.js and enables users to interactively enter values of network traffic features. The user-input data is sent to the processing hub via the Fast API based backend. The trained XGBoost model is used to oversee the received features to classify the network activity. Scikit-learn developed a model that takes the chosen parameters based on the NSL-KDD dataset, including the protocol type, service, flag, and network connection statistics. FastAPI processes the request for prediction, converts the request into a meaningful output and returns it to the visualization frontend interface. The scalable nature of the IDS through its modular design facilitates efficient operation, scalability, and easy maintenance, coupled with the provision of accurate and real-time intrusion detection and the provision of detailed visual interpretation of the results to the users.



[Fig - 3] System Architecture Diagram

4.3 Use Case Diagram:

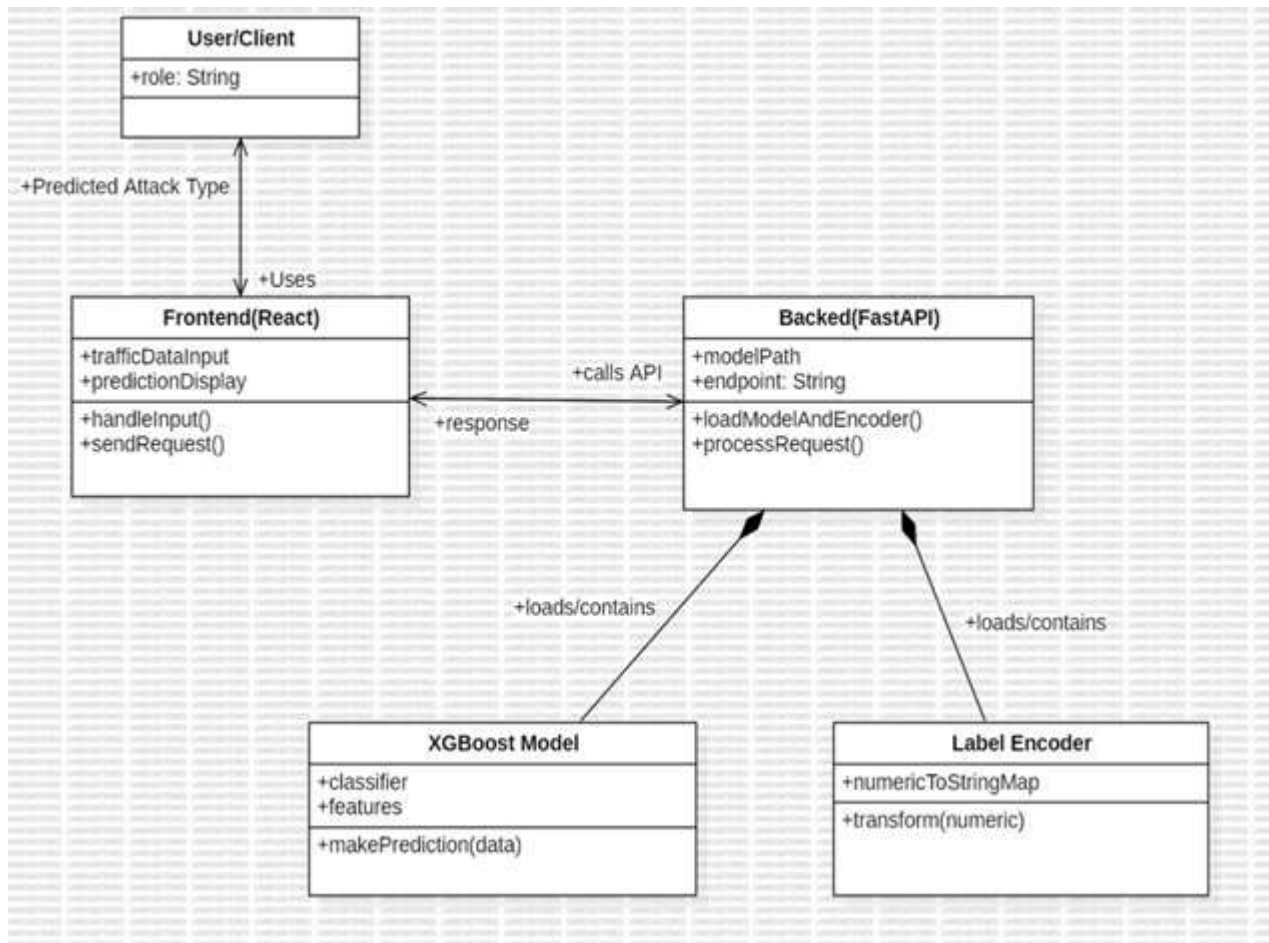
The use case diagrams create visual representations of software program actors and their interactions. They act as design elements that present developers with how human-machine interactions relate to personas at the same time, while establishing device responsibilities.



[Fig – 4] Use Case Diagram

4.4 Class diagram:

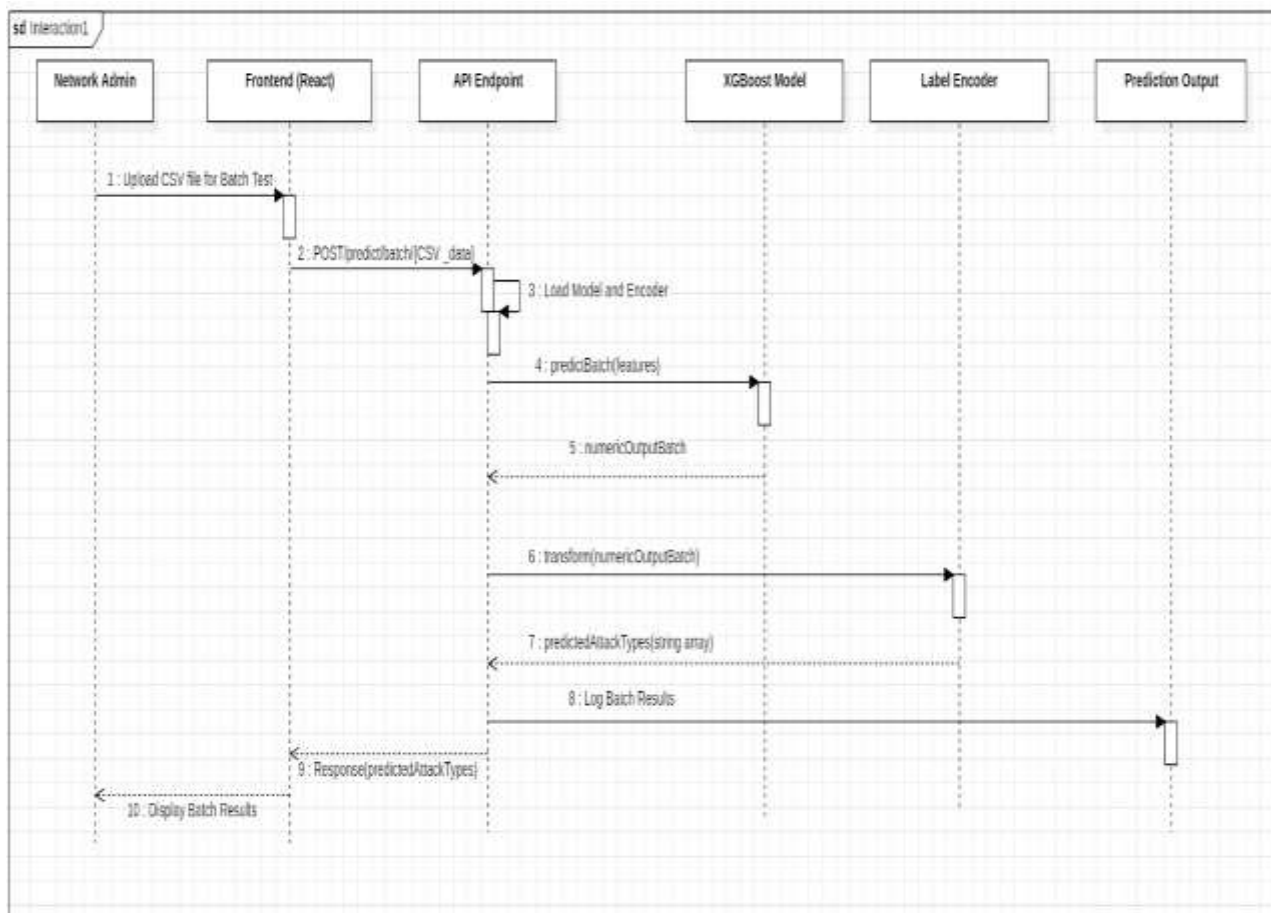
A UML elegance diagram serves as one core element that structures any object-oriented system. The sketch features an inflexible machine structure that displays educational and property-based initiatives together with procedural elements. Teaching within machines and the working functional code are expressed through this terminology.



[Fig – 5] Class Diagram

4.5 Sequence Diagram:

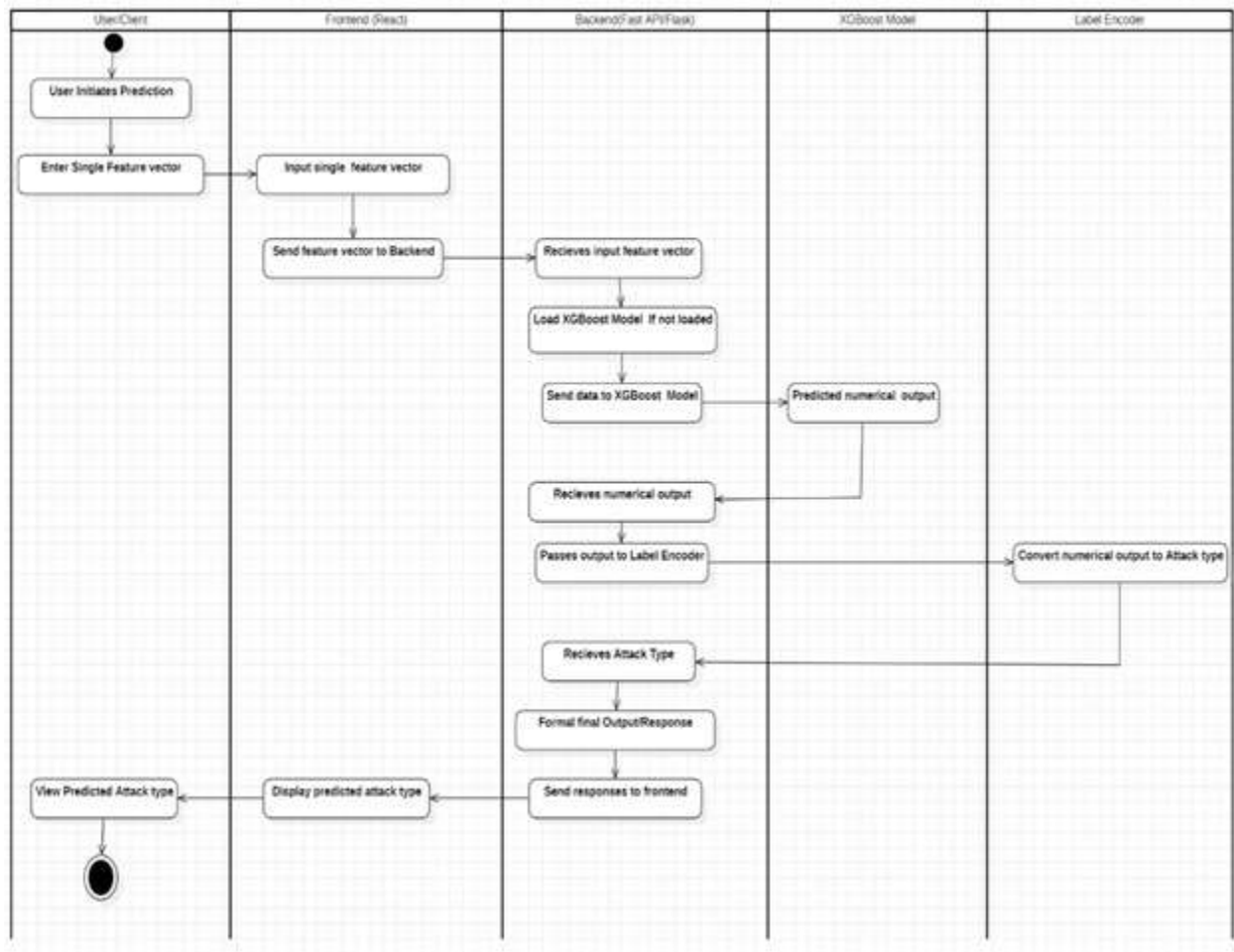
Among UML collection diagrams, you will find collection sequence diagrams, which scientists also name occurrence diagrams, that depict the sequential order of device interactions. Your designs include the strings that control character pieces and dialogue between characters and gadgets, which create conversations among these devices to meet a particular goal.



[Fig – 6] Sequence Diagram

4.6 Activity Diagram:

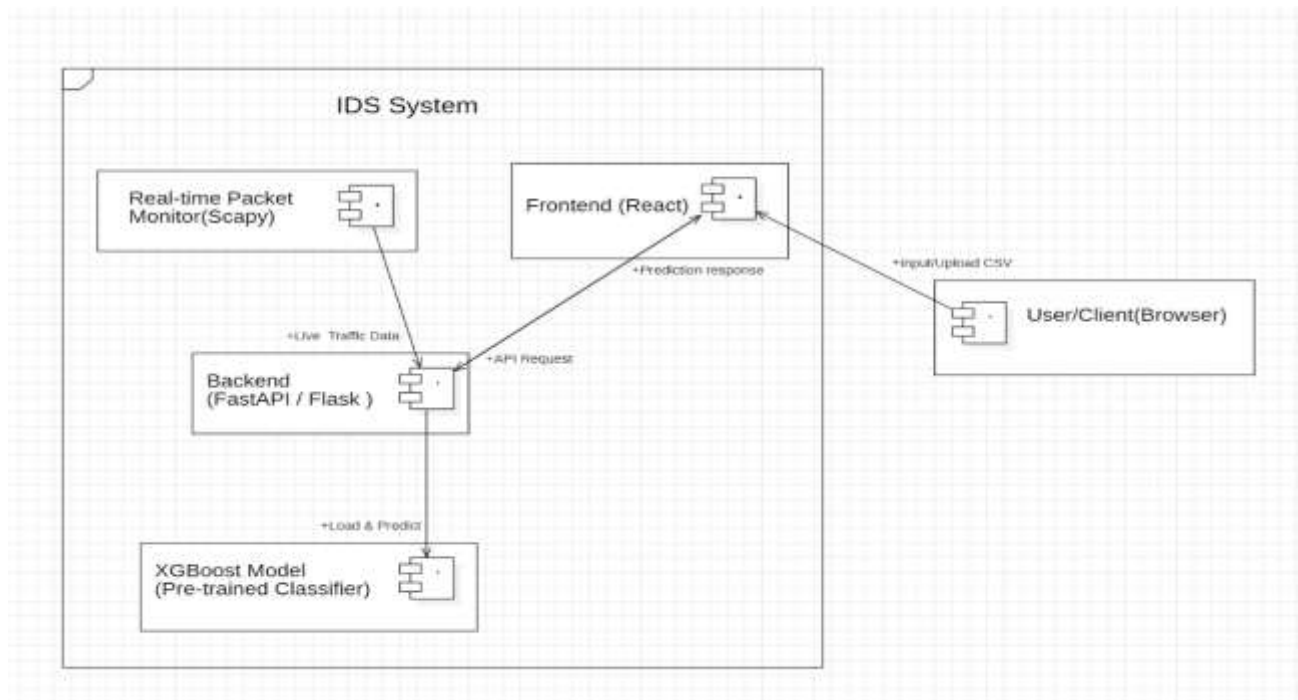
The UML activity diagram functions as a flowchart to depict all functional activities within a specific system. This shows the complete process from beginning to completion as it depicts various selection points and stages that seem connected between one step and another. The steps can appear in sequential order, following a timeline pattern, either starting from one timeline or operating simultaneously.



[Fig – 7] Activity Diagram

4.7 Component Diagram:

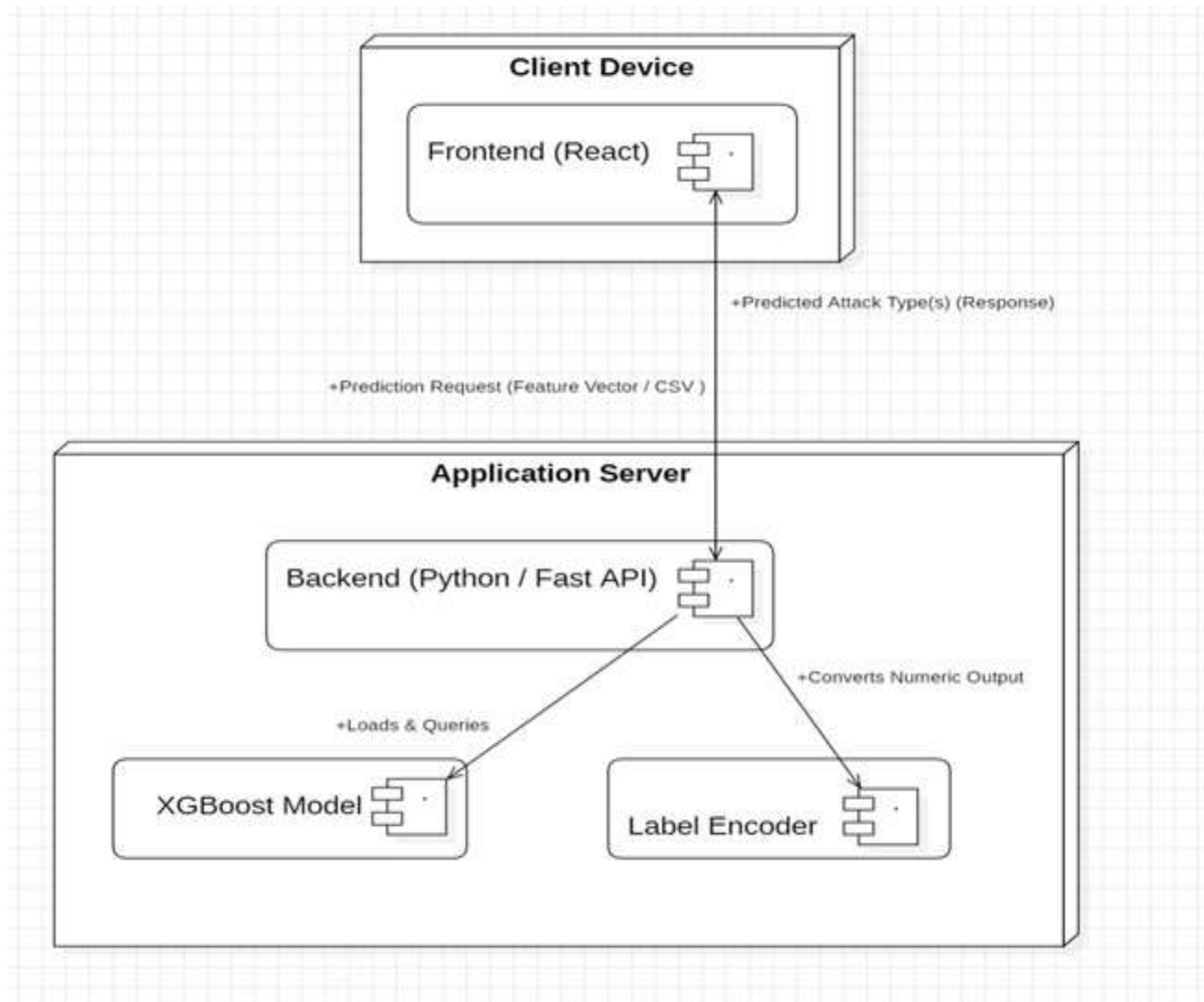
The UML component diagram represents the structural view of the IDS System by showing how each module interacts with others to perform intrusion detection. It highlights all major system components along with their communication links and responsibilities within the architecture. This diagram illustrates the complete setup from data collection to prediction delivery, showing how each part stays connected to support the overall workflow. The components work in a coordinated manner, either operating in sequence or functioning simultaneously to process user inputs, live traffic, backend logic, and model inference. Each module communicates through defined interfaces, ensuring smooth data flow between the user, frontend, backend, packet monitor, and the machine learning model.



[Fig – 8] Component Diagram

4.8 Deployment Diagram:

The deployment diagram illustrates how nodes will be distributed while presenting every element that will be present on these nodes. The simplest representation of this diagram shows the physical connectivity between nodes in distributed systems. These diagrams serve designers by assisting in software development for physical platforms.



[Fig – 9] Deployment Diagram

5. IMPLEMENTATION

5.1.1 ABOUT PYTHON

- Python can be used as a first programming language because of its readability, which may make it easy to learn and, therefore, it is adopted by both novices and mature programmers. Python is a language developed and tested quickly because of its clean syntax and dynamic typing. It is also characterized by automatic memory management and an enormous library system, thus making it perfect in terms of building an IDS. Python suits the development of the underlying infrastructure and the incorporation of the XGBoost classifier in network intrusion detection because of its abilities in the data analysis profession, machine learning, and web development.

5.1.2 APPLICATIONS OF PYTHON

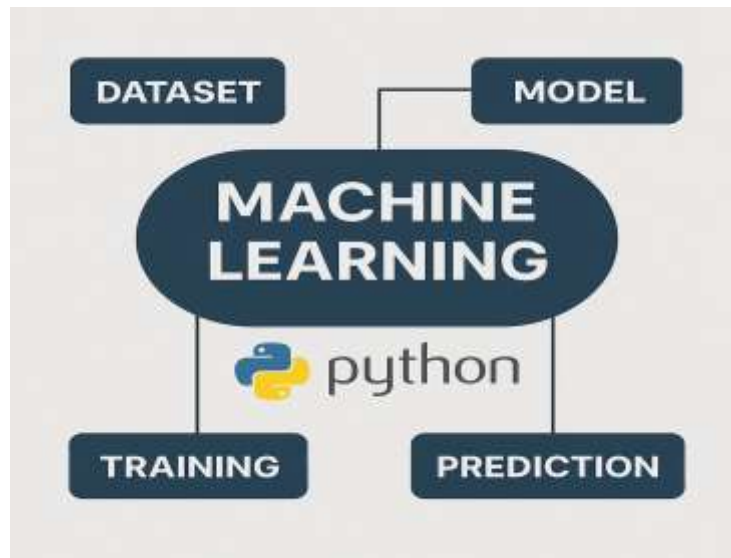
- Python has a wide range of applications outside the field of artificial intelligence, including data science and automation, and full-stack web development. Python is the language that the IDS project in this study uses as the framework of the back-end, with FastAPI, which facilitates interaction with the front-end. It is also helpful in the combination and implementation of the XGBoost machine learning model, which enables real-time identification of network traffic with the input characteristics provided by the NSL-KDD dataset. The flexibility of Python will give the system an efficient way of dealing with processing in the backend and machine learning.



[FIG-10] Applications of Python

5.1.3 PYTHON IN MACHINE LEARNING

- Python is the language of machine learning solution development since it has a rich library selection. With the IDS system, the entire machine learning process, such as the data preprocessing, model training, prediction, and merging with the frontend via FastAPI, can be conducted to perform real-time machine learning intrusion detection.



[FIG-11] Python In Machine Learning

5.1.4 PANDAS

- Python library used to work with network traffic data and allows exploration and preparation of data. The XGBoost model requires clean and transformed input, and Pandas makes the encoding of a categorical variable, feature selection, and normalization before training or prediction quite easy.

5.1.5 NUMPY

- NumPy serves to speed up numerical computations, and in particular when computing on multidimensional arrays of network traffic features. The computation in both model training and real-time network data can be done quickly through efficient matrix operations.

5.1.6 SCIKIT-LEARN

- The IDS makes use of Scikit-learn to use an XGBoost classifier, which is an effective framework to use in the process of supervised learning, training of the model and testing the accuracy on intrusion detection.

5.1.7 FRONTEND(REACT.JS)

- The IDS user interface is designed with the help of React.js. It has a component-based design that enables developers to create real-time interactive web applications. This interface allows users to enter the values of network features and get the immediate prediction of normal or intrusive traffic.

5.1.8 BACKEND(FLASK)

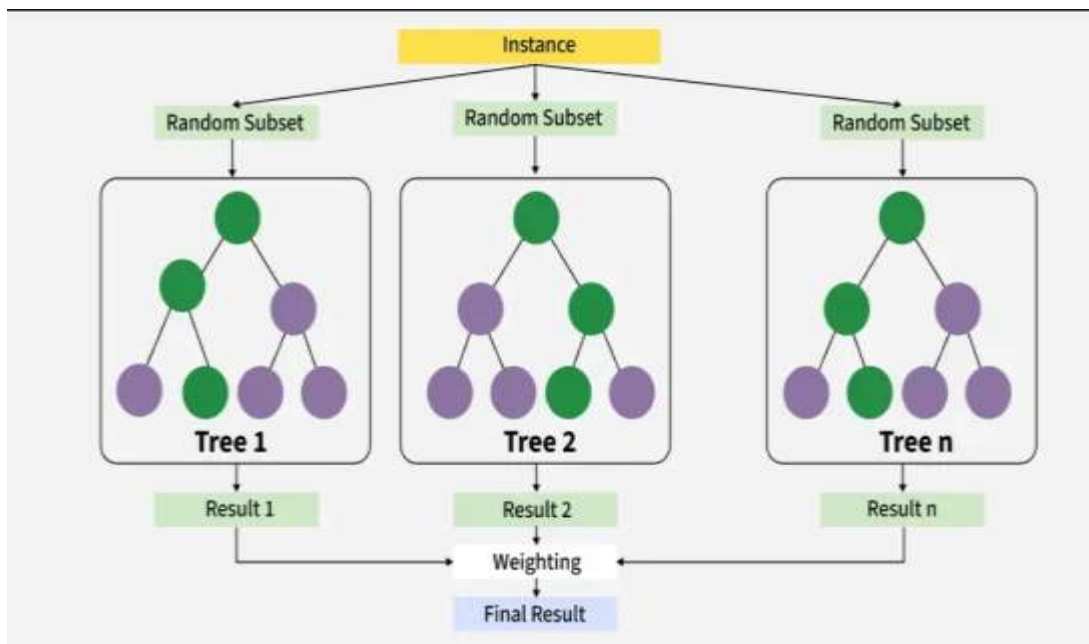
- The backend is a lightweight web framework called FastAPI and is used to manage the interaction between the frontend interface and the XGBoost model. Users input to the system, which processes the input, classifies it, and transfers the results back to the frontend. FastAPI is also efficient in the way it handles requests and has a smooth connection to the machine learning pipeline.

5.1.9 NSL-KDD DATASET

- The NSL-KDD data set offers an ideal network traffic data set on normal and intrusion connection samples. The presented dataset is the basis of training and evaluation of the XGBoost model, according to which the IDS will be capable of learning the trends in the network traffic and identifying anomalies.

5.2 XGBOOST MODEL

- XGBoost classifier is the predictive model of the IDS. It examines important features of the network, such as protocol type, service, connection flags and traffic statistics. These processed inputs allow the model to categorise the network connections either as normal or intrusion, and enhance real-time detection and general management of network security.

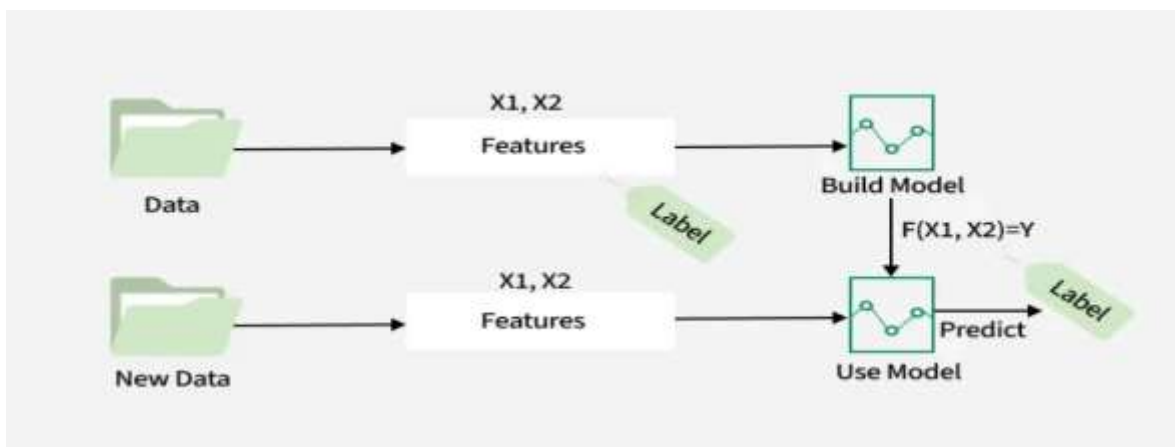


[FIG-12] XGBoost Model

❖ Machine Learning Model Overview:

The model aims to determine whether a product is going to be purchased based on its price and the behavioral peculiarities of the consumer.

- XGBoost is an ensemble method of learning that utilizes several weak learners to enhance the precision and stability of the model. XGBoost presents a good solution to the network intrusion detection problem because it is capable of dealing with high-dimensional and high-imbalanced data. This algorithm is appropriate to meet the specific needs of categorizing network traffic into normal and intrusive.



[FIG-13] Model Overview

5.3 DATASET DESCRIPTION

- The NSL-KDD dataset is used in the IDS project, and it consists of vital network traffic data, normal and attack connections with labelled classes. The data underlie the process of training and testing the machine learning model.

Input Features:

- **protocol Type:** Network protocol type (TCP, UDP, ICMP).
- **Service Network:** service on the destination (HTTP, SMTP, FTP, etc.).
- **Flag:** Flag of connection status (e.g., SF, S0).
- **Src Bytes:** Bytes sent by the source to a destination.
- **Dst Bytes:** Bytes that are transmitted by the destination to the source.
- **Duration:** Length of the network connection.
- **Other Statistical Features:** Count, number of failed logins, etc.

Target Variable: In each record, there is the appropriate label that shows whether the connection is a Normal one or an Intrusion.

Size of Dataset: Some 1.5 MB.

Sample Size: 25,000 records will be available to be used as a training, validation and testing sample.

Sample Data for the Problem Statement:

The network traffic feature data present was used to develop the underlying research problem. The IDS system uses measures on network parameters such as protocol type, service, connection flags, source bytes, destination bytes, duration, and other connection statistical parameters. These points of data are gathered by the system based on the NSL-KDD data set, and this enables the model to have access to actual and properly labelled past network traffic data. The data is then preprocessed by use of feature selection, coding of the categorical variables, as well as normalization, after which it is used to train the XGBoost classification model, which allows precise network connection detection and classification as to whether it is a normal or intrusive network connection.

duration	protocol_type	service	flag	src_bytes	dst_bytes	label
-0.11024922321249900	-0.12470615670462100	-0.686785207496691	0.7511112872365360	-0.00767859947168152	-0.004918644383724870	normal
-0.11024922321249900	2.2193118496444000	0.7814275260969030	0.7511112872365360	-0.007737369813364	-0.004918644383724870	normal
-0.11024922321249900	-0.12470615670462100	1.087305178928900	-0.7362346401101140	-0.00776224074056876	-0.004918644383724870	neptune
-0.11024922321249900	-0.12470615670462100	-0.442083085231092	0.7511112872365360	-0.0077227198161475000	-0.002891166974663830	normal
-0.11024922321249900	-0.12470615670462100	-0.442083085231092	0.7511112872365360	-0.007728341326091040	-0.004814199331375350	normal
-0.11024922321249900	-0.12470615670462100	1.087305178928900	-2.2235805674567600	-0.00776224074056876	-0.004918644383724870	neptune
-0.11024922321249900	-0.12470615670462100	1.087305178928900	-0.7362346401101140	-0.00776224074056876	-0.004918644383724870	neptune
-0.11024922321249900	-0.12470615670462100	1.087305178928900	-0.7362346401101140	-0.00776224074056876	-0.004918644383724870	neptune
-0.11024922321249900	-0.12470615670462100	1.2095562400617000	-0.7362346401101140	-0.00776224074056876	-0.004918644383724870	neptune
-0.11024922321249900	-0.12470615670462100	1.087305178928900	-0.7362346401101140	-0.00776224074056876	-0.004918644383724870	neptune
-0.11024922321249900	-0.12470615670462100	1.087305178928900	-2.2235805674567600	-0.00776224074056876	-0.004918644383724870	neptune
-0.11024922321249900	-0.12470615670462100	1.087305178928900	-0.7362346401101140	-0.00776224074056876	-0.004918644383724870	neptune
-0.11024922321249900	-0.12470615670462100	-0.442083085231092	0.7511112872365360	-0.007713350630241600	-0.0043588686388706800	normal
-0.11024922321249900	-0.12470615670462100	-0.686785207496691	0.7511112872365360	-0.00770634423586746	-0.004918644383724870	warezclient
-0.11024922321249900	-0.12470615670462100	0.2920232815657050	-0.7362346401101140	-0.00776224074056876	-0.004918644383724870	neptune
-0.11024922321249900	-0.12470615670462100	0.4143743426985050	-0.7362346401101140	-0.00776224074056876	-0.004918644383724870	neptune
-0.11024922321249900	-0.12470615670462100	-0.442083085231092	0.7511112872365360	-0.0077111360966274700	-0.001489862522307770	normal
-0.11024922321249900	-2.46887241630536500	-1.0538383908950900	0.7511112872365360	-0.007759174461872280	-0.004918644383724870	ipsweep
-0.11024922321249900	-0.12470615670462100	-0.442083085231092	0.7511112872365360	-0.007722549486331030	-0.004785458306945580	normal
-0.11024922321249900	-0.12470615670462100	-0.442083085231092	0.7511112872365360	-0.007703811096519220	-0.004625700879754080	normal
-0.11024922321249900	-0.12470615670462100	0.2308477509993050	-0.7362346401101140	-0.00776224074056876	-0.004918644383724870	neptune
-0.11024922321249900	-0.12470615670462100	1.087305178928900	-0.7362346401101140	-0.00776224074056876	-0.004918644383724870	neptune
-0.11024922321249900	-0.12470615670462100	-0.442083085231092	0.7511112872365360	-0.007719142490001610	-0.001958124507008120	normal
2.042559106866560	2.2193118496444000	0.7814275260969030	0.7511112872365360	-0.007737199464547530	-0.004882533120637490	normal
-0.11024922321249900	-0.12470615670462100	0.2308477509993050	-0.7362346401101140	-0.00776224074056876	-0.004918644383724870	neptune
0.08441348828207590	-0.12470615670462100	1.7802360151593000	0.7511112872365360	-0.007687798307770950	-0.0013324489076952800	normal

[Fig – 14] Sample Data from the Database

Import Libraries

- **Load Core Libraries:** Import Python libraries, which include pandas and numpy, used to manipulate and perform numerical procedures on data.
- **Machine Learning Libraries:** Import xgboost and train-test split of sklearn. Model selection to train and test the model.
- **Preprocessing Libraries:** Import: scaler, Label Encoder. If using categorical variables, it is necessary to encode them with Type sklearn. Preprocess and normalize the value of the features with a Type scaler.
- **Evaluation Libraries:** Import accuracy-score and confusion-matrix, and classification-report in sklearn. Metrics to evaluate the performance of the model.

```
train > xgboost_model.py > ...  
1  import pandas as pd  
2  import numpy as np  
3  from sklearn.preprocessing import LabelEncoder  
4  from sklearn.metrics import accuracy_score, classification_report, confusion_matrix  
5  from xgboost import XGBClassifier  
6  import joblib  
7
```

[Fig - 15] Libraries Importing

Preprocessing Data

- **Load Dataset:** Load the NSL-KDD data that has labelled network traffic data.
- **Select Features:** Select the top 10 important network features to model efficiently and in real-time.
- **Encode Categorical Variables:** Use Label Encoder to encode protocol type columns as well as service and flag columns into numeric values.
- **Normalized Features:** Standard Scaler is used to scale the numbers to enhance the model performance.

```

import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
train_path = "/Users/pratheek/Desktop/Major Project/data/KDDTrain+.txt"
test_path = "/Users/pratheek/Desktop/Major Project/data/KDDTest+.txt"
print("📂 Loading raw datasets ...")
columns = [
    'duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes',
    'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins',
    'logged_in', 'num_compromised', 'root_shell', 'su_attempted', 'num_root',
    'num_file_creations', 'num_shells', 'num_access_files', 'num_outbound_cmds',
    'is_host_login', 'is_guest_login', 'count', 'srv_count', 'error_rate',
    'srv_error_rate', 'error_rate', 'srv_error_rate', 'same_srv_rate',
    'diff_srv_rate', 'srv_diff_host_rate', 'dst_host_count',
    'dst_host_srv_count', 'dst_host_same_srv_rate', 'dst_host_diff_srv_rate',
    'dst_host_same_src_port_rate', 'dst_host_srv_diff_host_rate',
    'dst_host_error_rate', 'dst_host_srv_error_rate', 'dst_host_error_rate',
    'dst_host_srv_error_rate', 'label', 'difficulty_level'
]
train_df = pd.read_csv(train_path, names=columns)
test_df = pd.read_csv(test_path, names=columns)
print("Loaded datasets successfully!")
print(f"Train shape: {train_df.shape}, Test shape: {test_df.shape}")
train_df.drop(['difficulty_level'], axis=1, inplace=True)
test_df.drop(['difficulty_level'], axis=1, inplace=True)
cat_cols = ['protocol_type', 'service', 'flag']
encoder = LabelEncoder()
print("🔤 Encoding categorical columns:", cat_cols)
for col in cat_cols:
    train_df[col] = encoder.fit_transform(train_df[col])
    test_df[col] = encoder.transform(test_df[col])
scaler = StandardScaler()

```

[Fig -16] Preprocessing Data

Model Training

- **Split Data:** Divide the data into training (80%), testing (20).
- **Initialize Model:** Construct an XGBoost classifier with the most optimal hyperparameters.
- **Train Model:** The XGBoost model can be fitted on the training data using the following model.
- **Evaluate Model:** Determine the accuracy, confusion and classification on the test set.
- **Save Model:** Pickle the train model to be used at the backend as xgboost_ids_model.pkl.

```

xgb_model = XGBClassifier(
    n_estimators=400,
    learning_rate=0.1,
    max_depth=10,
    subsample=0.8,
    colsample_bytree=0.8,
    gamma=0.2,
    reg_lambda=1.5,
    random_state=42,
    use_label_encoder=False,
    eval_metric="mlogloss"
)

xgb_model.fit(X_train, y_train_enc)
print(" Model training complete!")

print("\n Evaluating model on test data ...")
y_pred = xgb_model.predict(X_test)

accuracy = accuracy_score(y_test_enc, y_pred)
print(f"Accuracy: {accuracy * 100:.2f}%\n")

print(" Classification Report:")
print(classification_report(y_test_enc, y_pred, zero_division=0))

print("Confusion Matrix:")
print(confusion_matrix(y_test_enc, y_pred))

```

[FIG-17] Model Training

Backend – Model Interaction

- **Load Model:** FastAPI backend loads XGBoost model that is saved on startup over pickle.
- **Get Requests:** Backend takes the input of network features made by the frontend through REST API calls.
- **Process Input:** Use the same preprocessing (encoding, scaling) as during training.
- **Predict Traffic:** Feed pass input to the XGBoost model to either classify as a Normal or an Intrusion.
- **Return Response:** This sends the prediction output back to the frontend, together with optional probability scores.

```
from fastapi import FastAPI
from pydantic import BaseModel
from typing import List
import numpy as np
import pandas as pd
import pickle
import joblib

app = FastAPI(
    title="Intrusion Detection System",
    description="FastAPI backend for IDS XGBoost model",
    version="1.0"
)

MODEL_PATH = "/Users/pratheek/Desktop/Major Project/backend/xgboost_ids_model.pkl"
ENCODER_PATH = "/Users/pratheek/Desktop/Major Project/backend/label_encoder.pkl"

try:
    with open(MODEL_PATH, "rb") as f:
        model = pickle.load(f)
        print("XGBoost model loaded successfully!")
except Exception as e:
    print(f"Failed to load XGBoost model: {e}")
    model = None

try:
    label_encoder = joblib.load(ENCODER_PATH)
    print("LabelEncoder loaded successfully!")
except Exception as e:
    print(f"Failed to load LabelEncoder: {e}")
    label_encoder = None

class TestInput(BaseModel):
    features: List[float]

class BatchTestInput(BaseModel):
    features_list: List[List[float]]

@app.get("/")
def root():
    return {"message": "Intrusion Detection System Backend is Running!"}

@app.get("/health")
def health_check():
    return {"status": "ok"}

@app.post("/predict")
def predict(data: TestInput):
    if model is None or label_encoder is None:
        return {"error": "Model or LabelEncoder not loaded."}

    try:
        features_array = np.array(data.features).reshape(1, -1)
        pred_num = model.predict(features_array)[0]
        pred_label = label_encoder.inverse_transform([pred_num])[0]
        return {"prediction": pred_label}
    except Exception as e:
        return {"error": str(e)}

@app.post("/predict-batch")
def predict_batch(data: BatchTestInput):
    if model is None or label_encoder is None:
        return {"predictions": [], "error": "Model or LabelEncoder not loaded."}

    try:
        features_array = np.array(data.features_list)
        pred_nums = model.predict(features_array)
        pred_labels = label_encoder.inverse_transform(pred_nums)
```

[FIG-18 & 19] Model Integration

Creating Encoders

- **Initialize Label Encoders:** Construct LabelEncoder objects of all categorical features (protocol type, service, flag).
- **Fit Encoders:** Fit all the encoders in the training column of respective features to learn to encode categories to numbers.
- **Transform Features:** Use the encoders to transform categorical values of both training and testing data.
- **Save Encoders:** Save encoder objects with pickle to be used in real-time backend prediction.

```
from sklearn.preprocessing import LabelEncoder
import pandas as pd
import pickle
df_train = pd.read_csv("/Users/pratheek/Desktop/Major Project/data/KDDTrain+.txt", header=None)

protocol_encoder = LabelEncoder()
service_encoder = LabelEncoder()
flag_encoder = LabelEncoder()

protocol_encoder.fit(df_train[1])
service_encoder.fit(df_train[2])
flag_encoder.fit(df_train[3])
import os
os.makedirs("/Users/pratheek/Desktop/Major Project/backend/encoders", exist_ok=True)

with open("/Users/pratheek/Desktop/Major Project/backend/encoders/protocol_type_encoder.pkl", "wb") as f:
    pickle.dump(protocol_encoder, f)

with open("/Users/pratheek/Desktop/Major Project/backend/encoders/service_encoder.pkl", "wb") as f:
    pickle.dump(service_encoder, f)

with open("/Users/pratheek/Desktop/Major Project/backend/encoders/flag_encoder.pkl", "wb") as f:
    pickle.dump(flag_encoder, f)
```

[Fig – 20] Encoders Creation

6. SOFTWARE TESTING

Software testing- It is a critical stage of development, and it precedes the finalization of software implementation. The primary objective of the stage is to establish whether the system matches the functional and non-functional requirements, as well as to identify and remove defects. This guarantees the reliability of the detection system of intrusion, accuracy, and real-time fully for the end users.

6.1.1 Unit Testing:

XGBoost Intrusion Detection Model:

Purpose: To test the machine learning model in terms of its accuracy to classify network traffic.

Tests:

- To test the prediction accuracy of the model, verification of the network traffic samples used in the NSL-KDD dataset must only be done with the verified samples.
- The system must provide the same outputs with the same input traffic data repeated several times.
- The model must be able to deal properly with extreme or abnormal network behaviours, e.g., uncharacteristically large packet sizes or uncharacteristic combinations of flags.
- The model must categorize traffic as per established attack signatures and normal connection patterns.

FastAPI Backend Processing:

Purpose: To check the functionality of the backend system to receive the request and process the data correctly.

Tests:

- The system will give accurate intrusion or normal prediction of various input network samples.
- The system must be able to process bad or partial input data.
- Backend will be expected to do appropriate processing of preprocessed input and provide predictions to the frontend.
- The endpoints of the API must be checked for stability under service failures and network outages.

6.1.2 Integration Testing:

Integration of Frontend with Backend and Model

Objective: The aim here is to have seamless interactions between the React.js frontend, FastAPI backend, and XGBoost model.

Tests:

- The system has to process real-time input for accurate end-to-end traffic classification.
- Data should flow seamlessly from the front-end input, through the back-end process, back to the front-end, and show the result.
- Model predictions update dynamically on changing network features
- The system should be tested for its response against delayed or failed API requests from the backend.

6.1.3 Functional Testing:

User Interaction and Interface Workflow:

Objective: The user will interact smoothly with the IDS platform and receive correct yet understandable predictions.

Tests:

- An exhaustive test would have to make sure that the interface presents all the correct classifications for all possible input combinations.
- Frontend form entries should be processed appropriately in the backend.
- Users should be allowed to input parameters of network traffic and get immediate classification results.
- Perform usability testing of the interface on a wide range of devices and various screen sizes.

6.1.4 Performance Testing:

Objective: To test system behaviours with real network traffic by logging on multiple users simultaneously.

Tests:

- Model response latency should be measured after the user inputs network features.
- Tests of speed and responsiveness with respect to multiple concurrent sessions.
- Analyse high-volume or complex traffic in terms of memory and CPU usage.
- Stress-test the system to make it able to handle larger volumes of users and immense volumes of traffic data.

6.1.5 Security Testing:

Data Security and Input Validation:

Objective: The system should be resistant to malicious input; sensitive data must be handled securely.

Tests:

- Test system behaviour when deliberately fed with incorrect or malicious network traffic inputs.
- Ensure that all user-submitted network data is secure from input to output.
- Verify that API communication between the frontend and backend is encrypted and resistant to tampering.
- Store API keys and other sensitive data securely in the backend and keep it out of reach of the frontend code.

Testing on our System:

This system was tested for various scenarios with different network traffic patterns and proved reliable in the correct identification of normal and intrusive connections. All in all, the input data were processed fast; stable performance and real-time results were obtained. In general, the IDS is highly accurate, robust, and secure for real-time applications in network intrusion detection.

6.2. RESULTS

- It used real-time network traffic data features and user input to generate very accurate, per-connection predictions from the IDS system. This model leveraged XGBoost on efficiently selected features, including protocol type, service, connection flags, source bytes, destination bytes, and duration. The React.js frontend provided the user with an interactive interface for network parameter input, while the FastAPI backend routed all communications with the machine learning model. Thus, the system could classify the traffic into Normal or Intrusion and showed real-time results. Therefore, this system would ensure smooth and highly reliable operation for network monitoring purposes.

The screenshot displays the 'Intrusion Detection System Dashboard' with the subtitle 'Network-based Cyber Threat Detection Dashboard'. The main heading is 'Intrusion Detection System Dashboard' with the description 'Visualize and analyze network traffic for potential attacks.' Below this is a form titled 'Upload or Enter Network Data' with two buttons: 'File Upload' and 'Manual Entry'. The form contains several input fields: a text field with '1', a dropdown menu with 'SF', a text field with '2', a text field with '0', a dropdown menu with 'tcp', a text field with '50', a text field with '40', a dropdown menu with 'http', a text field with '450', and a text field with '0'. A large blue 'Analyze' button is positioned at the bottom of the form.

[FIG-21] Result Dashboard



[FIG-22] Feature Importance



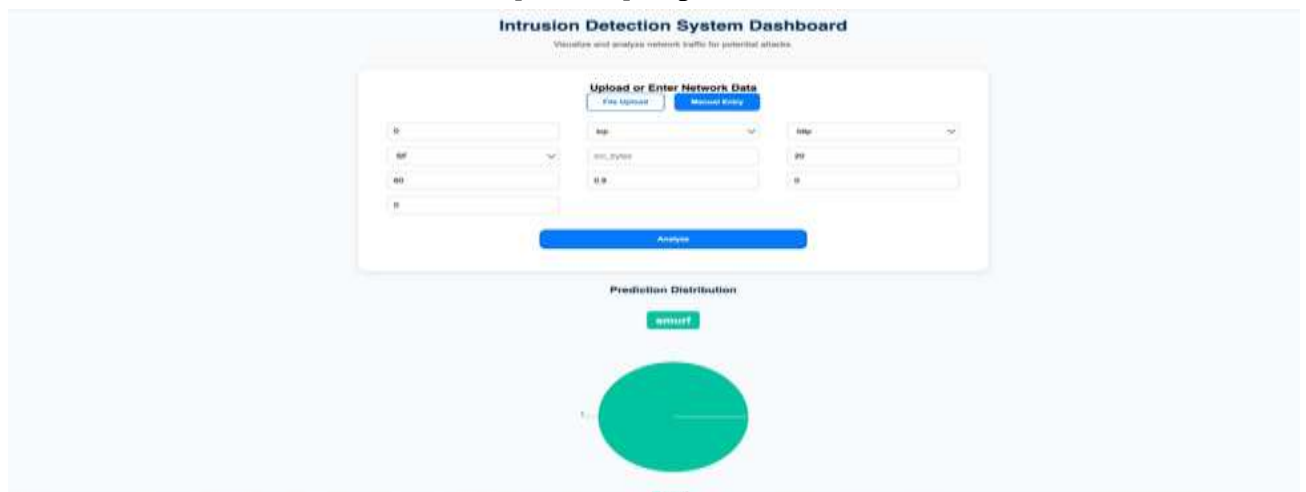
[FIG-23] Confusion Matrix



[FIG-24] Normal Prediction



[FIG-25] Neptune Prediction



[FIG-26] Smurf Prediction

7. CONCLUSION AND FUTURE SCOPE

Conclusion:

This AI-powered IDS detects network intrusions in real-time by analysing the features of the traffic with the intention of classifying connections correctly as normal or intrusive. Feature pre-processing, put together with the XGBoost machine learning model and the NSL-KDD dataset, empowers IDS to extract proper, data-driven insights about network security. Built on an intuitive frontend based on React.js, it has an efficient backend on FastAPI for handling all pre-processing, model interaction, and delivering predictions. This system enhances network security by providing instant, reliable alerts through automated detection, eliminating the need for continuous human surveillance.

Future Scope:

- **Interface in several languages:** Extension of the frontend for several languages is done to increase the usability among network administrators worldwide.
- **Advanced Feature Engineering:** Integrate more network traffic features or use behavioral patterns to detect complex or unseen attacks.
- **Enhancement of Machine Learning Models:** Representation of sequential and high-dimensional traffic data can be done with deep learning architectures such as LSTM or CNN for further improvement.
- **Integration with network monitoring tools:** It allows the IDS to interface with real-time network monitoring and logging systems for around-the-clock protection.
- **Offline and Edge Deployment:** IDS deployment is extended to edge devices for localized detection in networks that have limited access to the cloud or those with high latency.
- **IoT Network Security:** The proposed extensions to the IDS will implement monitoring for IoT devices through the detection of vulnerabilities in smart networks.
- **Security Collaboration Platform:** Where security administrators share different pieces of threat intelligence and intrusion patterns over various networks to learn from others.

8. REFERENCES

- [1] "Intrusion Detection using Machine Learning Techniques: An Experimental Comparison," by K.-A. Tait, J.S. Khan, F. Alqahtani, A.A. Shah, F.A. Khan, M. U. Rehman, W. Boulila, and J. Ahmad, 2021.
- [2] "Hybrid Framework for Intrusion Detection in Healthcare (ImmuneNet)," M. A. Kumaar et al., 2022.
- [3] Z. Li, "Toward Deep Learning based Intrusion Detection System," 2024.
- [4] B. Buyuktanir et al., "Federated Learning in Intrusion Detection: Advancements, Challenges, and Future Directions", 2025.
- [5] R. Kumar and M. Singh, "Hybrid Deep Neural Networks for Intrusion Detection," 2024.
- [6] "Anomaly Detection in Network Security using Autoencoders and Feature Fusion," L. Zhao and P. Chen, 2023.
- [7] S. Ahmed and R. Patel, "Blockchain-Enabled Intrusion Detection Framework for Secure IoT Networks," 2025.
- [8] H. Lee and D. Park, "Transfer Learning Approaches for Network Intrusion Detection," 2024.
- [9] "Optimized CNN Models for Lightweight Intrusion Detection for Edge Devices," J. Moreno and K. Gupta, 2023.
- [10] A. Sharma and L. Rodrigues, "A Survey on Explainable AI (XAI)-Based Next-Generation Intrusion Detection Systems," 2024.